

Note that the actual strings in the list aren't converted to lowercase; they're only sorted as if they were. Ned Batchelder provides more information about Unicode and code points in his talk "Pragmatic Unicode, or, How Do I Stop the Pain?" at <https://nedbatchelder.com/text/unipain.html>.

Incidentally, the sorting algorithm that Python's `sort()` method uses is Timsort, which was designed by Python core developer and "Zen of Python" author Tim Peters. It's a hybrid of the merge sort and insertion sort algorithms, and is described at <https://en.wikipedia.org/wiki/Timsort>.

Don't Assume Floating-Point Numbers Are Perfectly Accurate

Computers can only store the digits of the binary number system, which are 1 and 0. To represent the decimal numbers we're familiar with, we need to translate a number like 3.14 into a series of binary ones and zeros. Computers do this according to the IEEE 754 standard, published by the Institute of Electrical and Electronics Engineers (IEEE, pronounced "eye-triple-ee"). For simplicity, these details are hidden from the programmer, allowing you to type numbers with decimal points and ignore the decimal-to-binary conversion process:

```
>>> 0.3
0.3
```

Although the details of specific cases are beyond the scope of this book, the IEEE 754 representation of a floating-point number won't always exactly match the decimal number. One well-known example is 0.1:

```
>>> 0.1 + 0.1 + 0.1
0.30000000000000004
>>> 0.3 == (0.1 + 0.1 + 0.1)
False
```

This bizarre, slightly inaccurate sum is the result of *rounding errors* caused by how computers represent and process floating-point numbers. This isn't a *Python* gotcha; the IEEE 754 standard is a *hardware* standard